Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики
Кафедра теорії та технології програмування

ЗВІТ

з лабораторної роботи № 1

Паралельна обчислювальна геометрія: Реалізація алгоритму знаходження опуклої оболонки QuickHull із застосуванням системи ПАРКС у хмарному середовищі

Виконав:
студент групи ТТП-42
Бессарабов Володимир

Київ - 2026

1. Мета та ідея алгоритму

Мета роботи - реалізувати послідовний і паралельний QuickHull, розгорнути PARCS у Google Kubernetes Engine (GKE) та порівняти час обчислення для різної кількості воркерів.

QuickHull будує опуклу оболонку множини точок. Спочатку знаходяться крайні точки за координатою x та проводиться відрізок між ними. Він розділяє точки на верхню й нижню множини. Для кожного ребра алгоритм вибирає точку з найбільшим додатним векторним добутком відносно цього ребра. Обрана точка стає вершиною оболонки, а решта зовнішніх точок розподіляється між двома новими підзадачами. Рекурсія завершується, коли зовнішніх точок більше немає.

Паралельна реалізація розбиває детермінований вхід на неперетинні діапазони індексів. Кожен PARCS worker за спільним seed генерує свою частину точок, зберігає її в пам'яті та обчислює локальну оболонку. Координатор об'єднує лише вершини локальних оболонок і запускає QuickHull над цим малим набором:

$$\mathrm{Hull}(A \cup B) = \mathrm{Hull}(\mathrm{Hull}(A) \cup \mathrm{Hull}(B))$$

Підготовка воркерів та генерація їхніх діапазонів вимірюються окремо (`SetupMilliseconds`). Наведений у таблиці час охоплює обчислювальні раунди та обмін оболонками; він не включає створення воркерів.

2. PARCS-модуль і виправлення збірки

Модуль складається з `MainModule` і `WorkerModule`. Координатор надсилає команди ініціалізації та обчислення паралельно через канали PARCS, а потім перевіряє, що результат збігається з послідовним QuickHull. Для однакового тесту задаються `PointCount`, `Seed`, `Warmup` і `Repetitions`; `PointsNumber` задає кількість worker-ів.

Під час відновлення залежностей виникла проблема: PARCS8 вимагає `Parcs.Net` 10.1.0, якого немає у публічному NuGet feed. Ми зібрали пакет із зафіксованого вихідного коду PARCS8 і підключили його через локальний feed `artifacts/nuget`. Також `dotnet pack` завершувався помилкою `NU5026`, оскільки автоматичне пакування намагалося взяти DLL до завершення збірки. Це виправлено параметром `-p:GeneratePackageOnBuild=false` під час явного пакування. У контейнері Elasticsearch sink зроблено необов'язковим: патч вмикає його лише за заданого URL, тому PARCS запускається без окремого Elasticsearch і використовує консольне логування.

```
git clone https://github.com/alexeybogusevich/parcs8 third_party/parcs8
git -C third_party/parcs8 checkout d35d29f948cb1849a070967e681bc0df7f25d779
git -C third_party/parcs8 apply ../../patches/0001-optional-elasticsearch-logging.patch
dotnet pack third_party/parcs8/src/Parcs.Net/Parcs.Net.csproj -c Release `
  -o artifacts/nuget -p:GeneratePackageOnBuild=false
```

Зафіксована версія системи: PARCS8 commit `d35d29f948cb1849a070967e681bc0df7f25d779`. Реалізація лабораторної зберігається в репозиторії [QuickHullPARCS](https://github.com/valtosw/quickhull-parcs).

3. Налаштування хмарного середовища

Вимірювання виконано у GKE Standard, зона `europe-central2-a`, Kubernetes `1.36.4-gke.2377000`: один вузол пулу `default-pool` із міткою `parcs-role=master` і чотири вузли `workers` з міткою `parcs-role=worker`. Тип машини обох пулів - `e2-standard-2`. На кожному worker працював один PARCS daemon. Для спільних файлів використано NFS-Ganesha з PVC режиму `ReadWriteMany`.

Нижче наведено мінімальний порядок розгортання. Команди виконуються з кореня репозиторію у PowerShell. У проєкті Google Cloud має бути активований billing; замініть ідентифікатор проєкту та за потреби регіон/зону.

3.1. Проєкт, API та кластер

```
$PROJECT_ID = "<YOUR_PROJECT_ID>"
$REGION = "europe-central2"
$ZONE = "europe-central2-a"
$CLUSTER = "quickhull-parcs"
$REGISTRY = "$REGION-docker.pkg.dev/$PROJECT_ID/parcs"

gcloud auth login
gcloud config set project $PROJECT_ID
gcloud services enable compute.googleapis.com container.googleapis.com `
  artifactregistry.googleapis.com cloudbuild.googleapis.com --project=$PROJECT_ID
gcloud artifacts repositories create parcs --repository-format=docker `
  --location=$REGION --project=$PROJECT_ID
```

```
gcloud container clusters create $CLUSTER `
  --project=$PROJECT_ID --zone=$ZONE --num-nodes=1 `
  --machine-type=e2-standard-2 --node-labels=parcs-role=master --enable-ip-alias
gcloud container node-pools create workers `
  --cluster=$CLUSTER --project=$PROJECT_ID --zone=$ZONE --num-nodes=4 `
  --machine-type=e2-standard-2 --node-labels=parcs-role=worker
gcloud container clusters get-credentials $CLUSTER --zone=$ZONE --project=$PROJECT_ID

$PROJECT_NUMBER = gcloud projects describe $PROJECT_ID --format="value(projectNumber)"
$NODE_SA = "$PROJECT_NUMBER-compute@developer.gserviceaccount.com"
gcloud artifacts repositories add-iam-policy-binding parcs `
  --location=$REGION --project=$PROJECT_ID `
  --member="serviceAccount:$NODE_SA" --role=roles/artifactregistry.reader
```

3.2. Збірка образів і розгортання PARCS

Зберіть daemon, Host API та Portal з pinned PARCS8 source, а також образ послідовного baseline. Перед публікацією налаштуйте Docker для Artifact Registry:

```
docker build -f docker/parcs8.Dockerfile --target daemon `
  -t "$REGISTRY/daemon:lab" third_party/parcs8
docker build -f docker/parcs8.Dockerfile --target host `
  -t "$REGISTRY/host:lab" third_party/parcs8
docker build -f docker/parcs8.Dockerfile --target portal `
  -t "$REGISTRY/portal:lab" third_party/parcs8
dotnet publish modules/QuickHull.PARCS/QuickHull.PARCS.csproj -c Release
docker build -f docker/QuickHull.Sequential.Dockerfile `
  -t "$REGISTRY/quickhull-sequential:lab" .

gcloud auth configure-docker "$REGION-docker.pkg.dev"
docker push "$REGISTRY/daemon:lab"
docker push "$REGISTRY/host:lab"
docker push "$REGISTRY/portal:lab"
docker push "$REGISTRY/quickhull-sequential:lab"
```

Перед `dotnet publish` створіть локальний feed пакета `Parcs.Net` командою з попереднього розділу та додайте `artifacts/nuget` як NuGet source. Для NFS і PARCS виконайте:

```
$sources = dotnet nuget list source
if ($sources -notmatch 'parcs-local') {
    dotnet nuget add source "$PWD/artifacts/nuget" --name parcs-local
}
```

```
helm repo add nfs-ganesha-server-and-external-provisioner `
  https://kubernetes-sigs.github.io/nfs-ganesha-server-and-external-provisioner/
helm repo update
helm upgrade --install nfs-server `
  nfs-ganesha-server-and-external-provisioner/nfs-server-provisioner `
  --version 1.8.0 --namespace default --create-namespace `
  --values deploy/gcp/nfs-values.yaml
./deploy/scripts/deploy-gcp.ps1 -ProjectId $PROJECT_ID -Region $REGION -ImageTag lab
kubectl get nodes -o wide
kubectl get deployments,pods,pvc -o wide
```

У кластері працювали чотири PARCS daemon-поди, по одному на кожному worker-вузлі. Нижче показано стан вузлів і системних компонентів після розгортання.

```
kubectl get pods -o wide

NAME READY STATUS RESTARTS AGE IP NODE

parcs-daemon-f96c457fb-cbz8h 1/1 Running 0 17h 10.28.1.5 gke-quickhull-parcs-workers-768e9b9f-t1zd

parcs-daemon-f96c457fb-grw47 1/1 Running 0 17h 10.28.4.6 gke-quickhull-parcs-workers-768e9b9f-54z6

parcs-daemon-f96c457fb-pl6dr 1/1 Running 0 17h 10.28.3.7 gke-quickhull-parcs-workers-768e9b9f-17n5

parcs-daemon-f96c457fb-zp2qj 1/1 Running 0 16h 10.28.2.9 gke-quickhull-parcs-workers-768e9b9f-7jlq

---

parcs-database-85d4b7ccd8-xscn7 1/1 Running 0 17h 10.28.0.16 gke-quickhull-parcs-default-pool-986f8a24-9v3z

parcs-hostapi-5d8c869cd6-9dnd8 1/1 Running 0 17h 10.28.0.20 gke-quickhull-parcs-default-pool-986f8a24-9v3z

parcs-portal-d9b7dc47d-4dddr 1/1 Running 0 17h 10.28.0.19 gke-quickhull-parcs-default-pool-986f8a24-9v3z


4/4 PARCS daemon pods Ready — по одному на кожному worker-вузлі
```

Рисунок 1: Стан компонентів PARCS у GKE: чотири daemon-поди Ready на чотирьох worker-вузлах. Дані отримано командою kubectl get pods -o wide.

Скрипт розгортання створює Kubernetes Secret для пароля бази даних, підставляє registry у тимчасову копію маніфестів і очікує готовності чотирьох deployment-ів. Відкрийте Portal локально командою `kubectl port-forward service/parcs-portal 8080:80`, завантажте `QuickHull.PARCS.dll` та `QuickHull.PARCS.deps.json` з `modules/QuickHull.PARCS/bin/Release/net10.0/publish/`.

У Portal створіть job для assembly `QuickHull.PARCS` і класу `QuickHull.PARCS.MainModule`. Запустіть його з такими аргументами; повторіть запуск із `PointsNumber` від 1 до 4, не змінюючи решту параметрів:

```
{"PointsNumber":4,"PointCount":100000,"Seed":17,"Warmup":1,"Repetitions":5}
```

Послідовний baseline з такими самими параметрами запускається окремим Kubernetes Job:

```
./deploy/scripts/run-sequential.ps1 -ProjectId $PROJECT_ID -PointCount 100000 `
  -Seed 17 -Warmup 1 -Repetitions 5 -ImageTag lab
```

4. Методика і результати вимірювань

Для перевірки використано три нові детерміновані набори: 100 000 точок (seed 17), 1 000 000 точок (seed 2026) і 4 000 000 точок (seed 99). Для кожного набору виконано один послідовний Kubernetes Job і PARCS Job із `PointsNumber` 1, 2, 3 та 4. У кожному завданні було одне прогрівання і п’ять вимірюваних раундів. У таблиці наведено медіану цих п’яти раундів. Паралельний модуль перевіряє оболонку порівнянням із послідовним QuickHull; усі збережені оболонки збіглися точно за множиною координат.

Послідовний Job запускався на вузлі `workers` із лімітом 1 vCPU, як і daemon; його CPU request було зменшено до 500m, щоб Job міг запланувати ся поруч із daemon-ом на вузлі `e2-standard-2`. Setup PARCS наведений окремо й до часу обчислювальних раундів не входить.

Набір	Посл., мс	N=1	N=2	N=3	N=4	Найкраще	Верш.
100 тис. · seed 17	28,205	72,397	106,788	95,256	89,898	N=1 · 0,39×	29
1 млн · seed 2026	332,425	419,035	208,437	212,468	124,847	N=4 · 2,66×	35
4 млн · seed 99	1289,931	1367,523	859,555	478,041	333,767	N=4 · 3,86×	37

Таблиця 1: Медіанний час QuickHull у мілісекундах; для PARCS наведено час раунду без Setup.

Підготовка PARCS тривала 161-204 мс для 100 тис. точок, 134-194 мс для 1 млн і 208-246 мс для 4 млн (діапазон для N=1-4). Для найменшого набору паралельні накладні витрати перевищили обчислювальну економію: найкращим був N=1, але він усе одно повільніший за послідовну версію. Загалом найгірший виміряний режим - 100 тис. точок з N=2: прискорення 0,26×, тобто приблизно у 3,79 раза повільніше за послідовний baseline.

5. Залежність часу від кількості воркерів

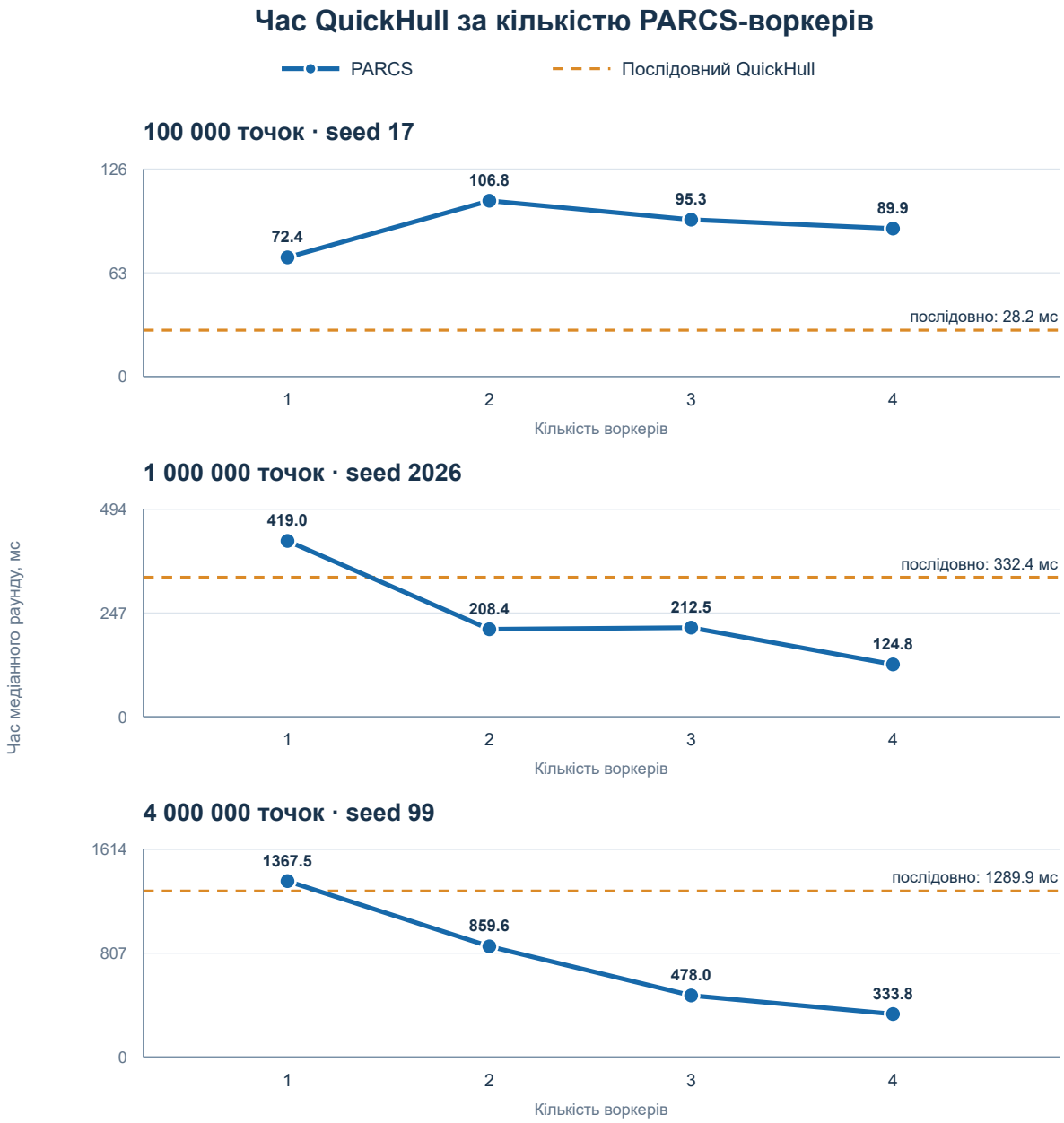


Рисунок 2: Для кожного розміру показано медіанний час PARCS для N=1-4 і горизонтальну лінію послідовного baseline. Менший час відповідає швидшому виконанню.

Для 1 млн точок найкращим у цьому запуску був N=4 - 124,847 мс або 2,66× прискорення. На найбільшому наборі N=4 дав найкращий результат у всьому експерименті: 333,767 мс проти 1289,931 мс послідовно, тобто 3,86×. Від N=1 до N=4 час зменшився приблизно у 4,10 раза. N=1 на 4 млн точок був трохи повільнішим за послідовний варіант через витрати PARCS-обміну без достатнього паралельного виграшу.

Висновок: розподілення QuickHull між незалежними діапазонами дає відчутне прискорення, коли розмір задачі достатній, щоб компенсувати створення worker-ів і комунікацію. Для малих наборів варто залишати послідовний режим або використовувати менше воркерів. Числа є результатом одного Kubernetes Job для кожної комбінації «набір × N» і п’яти внутрішніх раундів; медіана зменшує вплив окремого повільного раунду, але не усуває варіацію між повторними Job.